

How Heap Works

A Heap is a complete binary tree commonly used to implement priority systems efficiently.

In this project, a Max Heap was used because the patient with the highest severity must always be at the top.

Heap Operations

1. Insert Patient

When a new patient is added:

- The patient is inserted at the end of the heap.
- Heapify Up is performed.
- The node moves upward until the heap property is restored.

Time Complexity:

$O(\log n)$

2. Treat Next Patient

The patient with the highest priority is removed from the root.

Steps:

- Remove the root patient.
- Replace it with the last element.
- Perform Heapify Down to restore heap order.

Time Complexity:

$O(\log n)$

3. View Next Patient

The top patient is displayed without removing them.

Time Complexity:

$O(1)$

4. Update Severity

If a patient's severity changes:

- The severity value is updated.
- Heapify Up or Heapify Down is applied depending on the new value.

Time Complexity:

$O(\log n)$

Why Heap Is Efficient

Heap is efficient because it keeps the highest priority element at the root. Insertion and deletion operations are performed in logarithmic time. This makes heap suitable for systems such as emergency rooms, CPU scheduling, task management systems, and network routing.

Comparison with Other Structures

Structure	Insert	Delete Highest Priority	Search	Notes
Heap	$O(\log n)$	$O(\log n)$	$O(n)$	Best for priority systems
Array	$O(1)$	$O(n)$	$O(n)$	Simple but inefficient
Linked List	$O(1)$	$O(n)$	$O(n)$	Requires traversal
BST	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg	More complex
AVL Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	Balanced but harder

Manual Heap vs STL priority_queue

Manual Heap	STL priority_queue
Implemented from scratch	Built-in STL container
Helps understand heap internals	Easier and faster to use
Requires manual heapify operations	Automatically maintains heap
Better for learning data structures	Better for real applications

Time Complexity Comparison

Operation	Manual Heap	STL priority_queue
Insert	$O(\log n)$	$O(\log n)$
Remove Top	$O(\log n)$	$O(\log n)$
View Top	$O(1)$	$O(1)$
Ease of Use	Harder	Easier